SIMATS ENGINEERING

DESIGN AND ANALYSIS OF ALGORITHMS

CO3-Assignment-3

Student-A Name: A. Abhishek Reddy

Student-B A. Vikranth Reddy

Rg no:192424192

Course Code: CSA0601

Course Name: DESIGN AND ANALYSIS OF ALGORITHMS

1. Introduction to Design and Analysis of Algorithms

An algorithm is a finite sequence of well-defined steps used to solve a particular computational problem. It accepts input, processes the input using a specific method, and produces the required output. Algorithms are important in computer science because they provide a systematic way to solve problems efficiently.

The Design and Analysis of Algorithms subject focuses on developing correct algorithms and evaluating their performance. An algorithm should not only produce the correct answer but should also use computational resources efficiently. These resources mainly include execution time and memory space.

When the size of the input increases, the amount of work performed by an algorithm may also increase. Therefore, algorithm analysis helps us understand how efficiently an algorithm works for small, medium, and large inputs.

2. Objectives of the Assignment

The main objectives of this assignment are:

1. To understand the Divide-and-Conquer algorithm design technique.
2. To divide large problems into smaller and manageable subproblems.
3. To understand how recursive algorithms operate.
4. To compare different algorithms based on time and space complexity.

5. To study how decomposition quality affects algorithm performance.
6. To implement and validate different algorithmic solutions.
7. To measure comparisons, execution time, and recursion depth.
8. To understand the difference between theoretical and practical performance.
9. To develop independent problem-solving skills through pair work.
10. To evaluate algorithm designs using mathematical and experimental evidence.

3. Divide-and-Conquer Technique

Divide-and-Conquer is an important algorithm design technique in which a large problem is divided into smaller subproblems of the same or related type. Each subproblem is solved independently, and the results are combined to obtain the final solution.

It generally contains three main steps:

Step	Description
Divide	Break the original problem into smaller subproblems.
Conquer	Solve each subproblem directly or recursively.
Combine	Combine the partial results to obtain the final answer.

4. General Structure of Divide-and-Conquer

The general structure can be represented as:

1. Receive the original input.
2. Check whether the input is small enough to solve directly.
3. If it is a small problem, return the direct solution.
4. Otherwise, divide the input into smaller subproblems.
5. Recursively solve the subproblems.
6. Combine their results.
7. Return the final result.

5. Advantages of Divide-and-Conquer

- It reduces a large problem into smaller problems.
- It can reduce the total number of operations.
- It supports recursive problem-solving.
- It is useful for sorting, searching, multiplication, and geometric problems.
- It provides a clear structure for analysing algorithm performance.

1. Meaning of Algorithm Analysis

Algorithm analysis is the process of measuring the efficiency of an algorithm. It helps determine how much time and memory are required to solve a problem.

Two algorithms may produce the same output, but their execution times may be different. For example, one sorting algorithm may complete quickly for a large dataset, while another may take more time.

Therefore, analysing algorithms is important before selecting an appropriate solution.

2. Time Complexity

Time complexity describes how the number of operations performed by an algorithm increases with input size.

It is commonly represented using Big-O notation.

Complexity	**Meaning**	**Example**
(O(1))	Constant time	Accessing an array element
(O(\log n))	Logarithmic time	Binary Search
(O(n))	Linear time	Linear Search
(O(nlog n))	Linearithmic time	Merge Sort
(O(n^2))	Quadratic time	Simple nested-loop sorting

(O(2^n))	Exponential time	Straightforward Subset-Sum enumeration

3. Space Complexity

Space complexity measures the additional memory required by an algorithm during execution.

It may include:

- Variables and temporary storage.
- Arrays and other data structures.
- Recursive function call stacks.
- Intermediate results.
- Memory used during sorting or merging.

An algorithm with low time complexity may still require a large amount of memory. Therefore, both time and space complexity must be considered.

4. Recurrence Relation

A recurrence relation describes the running time of a recursive algorithm in terms of smaller input sizes.

For example, an algorithm that divides a problem into two equal parts and performs linear work to combine the results can be represented as:

[T(n)=2T(n/2)+O(n)]

Here:

- (2T(n/2)\) represents two recursive subproblems.
- (O(n)\) represents the work required to combine the results.
- (T(n)\) represents the total running time.

Solving the recurrence helps determine the overall time complexity.

5. Experimental Analysis

Experimental analysis measures the actual performance of an implemented algorithm.

The following information can be recorded:

Measurement	Purpose
Input size	Shows the amount of data processed
Comparison count	Measures comparisons performed
Execution time	Measures practical running time
Recursion depth	Measures the maximum recursive call depth
Memory usage	Estimates additional storage requirements

Theoretical analysis predicts the growth of computational work, while experimental analysis shows how the implementation behaves in practice.

Q1 – DECOMPOSITION AND ELIMINATION IN ORDERED DATA

PART A: FINDING MAXIMUM AND MINIMUM

1. Introduction

Finding the maximum and minimum values in an array is a basic problem in algorithm design. The maximum value is the largest element in the array, while the minimum value is the smallest element.

A simple method is to find the maximum and minimum separately. However, this method may perform more comparisons than necessary.

The Divide-and-Conquer approach reduces unnecessary comparisons by dividing the array into smaller parts, finding the maximum and minimum of each part, and combining the results.

2. Problem Definition

Given an array containing (n) elements, the task is to find:

- The largest element.
- The smallest element.

For example, consider the array:

[A=[12,5,8,21,3,17,9,14] \]

The maximum element is:

[Maximum=21 \]

The minimum element is:

[Minimum=3]

3. Independent Maximum and Minimum Method

In the independent method, the algorithm finds the maximum value first and then finds the minimum value.

For finding the maximum, the algorithm compares each element with the current maximum.

For finding the minimum, it compares each element with the current minimum.

For an array of (n) elements:

- Maximum requires (n-1) comparisons.
- Minimum requires (n-1) comparisons.

Therefore, the total number of comparisons is:

[(n-1)+(n-1)=2n-2]

The time complexity is:

[O(n)]

4. Divide-and-Conquer Approach

In the Divide-and-Conquer method, the array is divided into two smaller parts.

Each part is processed recursively to find its maximum and minimum values.

The results are combined as follows:

[Maximum=max(Maximum_L,Maximum_R) \]

[Minimum=min(Minimum_L,Minimum_R) \]

Here:

- (Maximum_L\) represents the maximum value in the left part.
- (Maximum_R\) represents the maximum value in the right part.
- (Minimum_L\) represents the minimum value in the left part.

- (Minimum_R\) represents the minimum value in the right part.

5. Decomposition Example

Consider:

[A=[12,5,8,21,3,17,9,14] \]

First divide the array into two halves.

Left half	Right half
12, 5, 8, 21	3, 17, 9, 14

Divide each half again.

Subproblem	Elements
A1	12, 5
A2	8, 21
A3	3, 17
A4	9, 14

Find the maximum and minimum of each pair.

Subproblem	Maximum	Minimum
A1	12	5
A2	21	8
A3	17	3
A4	14	9

Now combine the results.

Left half:

[Maximum_L=max(12,21)=21]

[Minimum_L=min(5,8)=5]

Right half:

[Maximum_R=max(17,14)=17]

[Minimum_R=min(3,9)=3]

Final result:

[Maximum=max(21,17)=21]

[Minimum=min(5,3)=3]

Final Answer: Maximum = 21, Minimum = 3

MAXIMUM AND MINIMUM: ALGORITHM AND ANALYSIS

1. Language-Independent Pseudocode

ALGORITHM FindMaxMin(A, low, high)

INPUT:

- A – array containing elements.
- low – starting index.
- high – ending index.

OUTPUT:

- Maximum and minimum elements in the given range.

BEGIN

1. IF low = high THEN
    - RETURN A[low] as both maximum and minimum.
2. END IF
3. IF high = low + 1 THEN
    - IF A[low] greater than A[high] THEN
        - RETURN A[low] as maximum and A[high] as minimum.

- ELSE
    - RETURN A[high] as maximum and A[low] as minimum.
- END IF

4. END IF

5. mid ← FLOOR((low + high) / 2)

6. leftResult ← FindMaxMin(A, low, mid)

7. rightResult ← FindMaxMin(A, mid + 1, high)

8. maximum ← MAX(leftResult.maximum, rightResult.maximum)

9. minimum ← MIN(leftResult.minimum, rightResult.minimum)

10. RETURN maximum and minimum.

END

2. Explanation of the Pseudocode

The algorithm first checks whether the current subarray contains only one element. If it does, that element is both the maximum and minimum.

If the subarray contains two elements, the algorithm compares them once and identifies the larger and smaller values.

For larger subarrays, the algorithm calculates the middle position and divides the array into two parts.

It recursively finds the maximum and minimum values in both parts. Finally, it compares the two maximum values and the two minimum values to produce the result.

3. Comparison Analysis

For an even number of elements, using pairwise comparison and recursive combination, the number of comparisons is:

[C(n)=frac{3n}{2}-2]

for the standard balanced tournament method when \(n\) is a power of two.

For example, if (n=8):

[C(8)=frac{3(8)}{2}-2]

[=12-2=10]

The independent method requires:

[2n-2=2(8)-2=14]

Therefore, the Divide-and-Conquer tournament method uses 10 comparisons instead of 14.

4. Comparison Table

Input size	Independent method	Tournament method
2	2	1
4	6	4
8	14	10
16	30	22
32	62	46

The tournament counts shown use the standard pairwise strategy for these even, power-of-two input sizes.

5. Complexity Analysis

The recurrence for the balanced method is:

[T(n)=2T(n/2)+O(1)]

The algorithm creates two recursive subproblems, each containing approximately half of the original elements. Only a constant number of comparisons are needed to combine the two results.

Using the Master Theorem:

[T(n)=O(n)]

The additional recursion stack requires:

[O(log n)]

space for a balanced recursive implementation, excluding any additional storage used to represent returned results.

6. Advantages

- Reduces the number of comparisons.
- Uses a systematic recursive approach.
- Finds both maximum and minimum values together.
- Demonstrates the Divide-and-Conquer technique.
- Provides a clear comparison between two algorithm designs.

EXPERIMENTAL VALIDATION OF MAXIMUM AND MINIMUM

1. Purpose of Experimental Validation

Experimental validation is used to check whether the theoretical comparison count matches the actual implementation.

The algorithm is tested using different input sizes. A comparison counter is included to count how many times two array elements are compared.

The observed results are compared with the expected results.

2. Dataset Design

The experiment should use different types of datasets.

Dataset	**Example**	**Purpose**
Increasing	1, 2, 3, 4, 5	Tests ascending data
Decreasing	9, 8, 7, 6, 5	Tests descending data
Random	8, 3, 12, 5, 1	Tests mixed values
Equal elements	7, 7, 7, 7	Tests duplicate values
Negative values	-4, -9, -1, -7	Tests negative inputs

Testing different datasets helps identify whether the algorithm behaves correctly under different input conditions.

3. Experimental Procedure

1. Select an input size.
2. Generate an array containing the required number of elements.
3. Initialise the comparison counter to zero.
4. Execute the independent maximum and minimum method.
5. Record the comparison count.
6. Execute the Divide-and-Conquer method.
7. Record its comparison count.
8. Compare both outputs to verify correctness.
9. Repeat the experiment for larger input sizes.
10. Record all observations in a table.

4. Observation Table

The following table shows theoretical comparison counts for selected input sizes.

Input size	Independent comparisons	Tournament comparisons	Reduction
8	14	10	4
16	30	22	8
32	62	46	16
64	126	94	32
128	254	190	64

5. Interpretation of Results

The table shows that the tournament method performs fewer comparisons than independently finding the maximum and minimum.

As the input size increases, the number of comparisons also increases. However, the tournament method continues to maintain a lower comparison count.

The experimental result should be checked against the theoretical formula. Any difference may occur because of the input size, the handling of an odd number of elements, or the exact comparison-counting method used in the implementation.

6. Student Responsibilities

Student	Responsibility
Student A	Explain decomposition, base cases, combination, and mathematical comparison analysis.
Student B	Design datasets, instrument the program, record comparisons, and compare observed results with theoretical predictions.
Joint	Implement, test, and verify the final output.

7. Conclusion

The Divide-and-Conquer tournament method is an efficient approach for finding the maximum and minimum values of an array. It divides the array into smaller parts, solves each part, and combines the results.

Compared with independently finding the maximum and minimum, it reduces the number of comparisons. The experiment helps confirm the theoretical advantage of the method.

PART B: BINARY SEARCH

1. Introduction

Binary Search is an efficient searching algorithm used to locate a target element in a sorted array. It works by repeatedly dividing the search range into two halves.

Unlike Linear Search, which checks elements one by one, Binary Search eliminates half of the remaining search range after each comparison.

Binary Search is suitable only when the input data is sorted or when an equivalent ordering structure is available.

2. Problem Definition

Given a sorted array and a target value, the task is to determine whether the target exists in the array.

If the target is present, the algorithm returns its position. Otherwise, it returns NOT_FOUND.

Consider the sorted array:

[A=[3,7,11,15,19,23,27,31,35]]

Let the target be:

[Target=23]

The algorithm checks the middle element and decides which half may contain the target.

3. Working Principle

Binary Search follows these steps:

1. Set the lower boundary to the first array position.
2. Set the upper boundary to the last array position.
3. Calculate the middle position.
4. Compare the middle element with the target.
5. If both are equal, return the middle position.
6. If the target is smaller, continue in the left half.
7. If the target is larger, continue in the right half.
8. Repeat until the target is found or the search range becomes empty.

4. Example

Array:

[[3,7,11,15,19,23,27,31,35]]

Target:

[23]

Step	Low	High	Middle	Middle value	Decision
1	0	8	4	19	Search right

2	5	8	6	27	Search left
3	5	5	5	23	Target found

The target value 23 is found at index 5.

5. Elimination Logic

The important feature of Binary Search is that it eliminates a portion of the search space after every comparison.

If the middle element is smaller than the target, all elements to the left of the middle position can be eliminated because the array is sorted.

Similarly, if the middle element is greater than the target, all elements to the right can be eliminated.

This decision is correct because the sorted order guarantees that the eliminated section cannot contain the target.

BINARY SEARCH: PSEUDOCODE AND COMPLEXITY

1. Language-Independent Pseudocode

ALGORITHM BinarySearch(A, low, high, target)

INPUT:

- A – sorted array.
- low – starting index of the search range.
- high – ending index of the search range.
- target – element to be searched.

OUTPUT:

- Position of the target, or NOT_FOUND.

BEGIN

1. WHILE low ≤ high DO
2. mid ← FLOOR((low + high) / 2)

3. IF A[mid] = target THEN
    - RETURN mid.
4. ELSE IF target is less than A[mid] THEN
    - high ← mid - 1.
5. ELSE
    - low ← mid + 1.
6. END IF
7. END WHILE
8. RETURN NOT_FOUND.

END

2. Explanation

The algorithm starts with the complete sorted array as the search range. It calculates the middle position and compares the middle element with the target.

If the values are equal, the search is completed successfully.

If the target is smaller than the middle element, the algorithm changes the upper boundary to mid - 1. This removes the right portion of the array.

If the target is larger, the lower boundary becomes mid + 1. This removes the left portion.

The process continues until the target is found or the lower boundary becomes greater than the upper boundary.

3. Successful Search

A successful search occurs when the target element is present in the array.

Example:

[A=[10,20,30,40,50,60,70]]

Target:

[Target=50]

The algorithm checks the middle element, compares it with 50, and continues in the appropriate half until the target is found.

4. Unsuccessful Search

An unsuccessful search occurs when the target is not present in the array.

Example:

[A=[10,20,30,40,50,60,70]]

Target:

[Target=45]

The algorithm repeatedly eliminates the half that cannot contain the target. Finally, the search range becomes empty, and the algorithm returns NOT_FOUND.

5. Complexity Analysis

After every comparison, the search range becomes approximately half its previous size.

The recurrence can be represented as:

[T(n)=T(n/2)+O(1)]

Therefore, the time complexity is:

[boxed{O(log n)}]

Case	**Time complexity**	**Explanation**
Best case	(O(1))	Target is found at the first middle position.
Average case	(O(log n))	Several halves are eliminated.
Worst case	(O(log n))	Search continues until the range becomes empty or the target is found at the final stage.

The iterative pseudocode uses (O(1)) auxiliary space because it does not create recursive calls.

BINARY SEARCH: EXPERIMENTAL INVESTIGATION

1. Purpose

The purpose of this experiment is to observe how the number of comparisons changes when the input size increases.

Both successful and unsuccessful search cases must be tested. The comparison count is recorded to demonstrate how repeated elimination reduces the amount of work.

2. Experimental Datasets

The following sorted arrays can be used:

Dataset	Input size	Array
A	8	1, 2, 3, 4, 5, 6, 7, 8
B	16	1 to 16
C	32	1 to 32
D	64	1 to 64
E	128	1 to 128

For each dataset, select:

- A target that is present in the array.
- A target that is not present in the array.

3. Experimental Procedure

1. Create a sorted array.
2. Select the target value.
3. Initialise the comparison counter to zero.
4. Perform Binary Search.
5. Increase the counter whenever the middle element is compared with the target.
6. Record whether the target is found.
7. Repeat with an unsuccessful target.

8. Increase the input size and repeat the experiment.
9. Compare the observed number of comparisons.

4. Theoretical Observation Table

The following table gives approximate upper-bound comparison counts for a standard Binary Search implementation. Exact counts depend on the middle-position convention and the selected target.

Input size	Approximate maximum comparisons
8	4
16	5
32	6
64	7
128	8
256	9
512	10
1024	11

5. Interpretation

The comparison count increases slowly compared with the input size.

For example, increasing the input size from 128 to 256 does not double the number of comparisons. Only approximately one additional comparison is required in the worst-case search depth.

This happens because Binary Search eliminates half of the remaining elements after every comparison.

6. Advantages

- It is faster than Linear Search for large sorted datasets.

- It requires fewer comparisons.
- It follows a clear elimination strategy.
- It has logarithmic time complexity.
- It is useful in dictionaries, databases, and sorted collections.

7. Limitations

- The array must be sorted.
- It is not directly suitable for unsorted data.
- Maintaining sorted data may require additional work.
- It is less suitable for data structures without efficient middle-element access.

8. Student Responsibilities

Student	Responsibility
Student A	Design successful and unsuccessful test cases, record comparisons, and present experimental evidence.
Student B	Explain the decision rule, elimination logic, and termination condition.
Joint	Derive and implement the search algorithm and verify the output.

9. Conclusion

Binary Search is an efficient searching technique that repeatedly divides the search range into two halves. Its logarithmic time complexity makes it suitable for large sorted datasets.

The experiment demonstrates that the number of comparisons increases slowly as the input size grows. This confirms the advantage of eliminating half of the search space after each comparison.

Q2 – THE QUALITY OF DECOMPOSITION IN SORTING

INTRODUCTION TO MERGE SORT AND QUICK SORT

1. Introduction

Sorting is the process of arranging data in a particular order, such as ascending or descending order. Sorting is widely used in databases, searching systems, record management, and data processing applications.

Merge Sort and Quick Sort are two important Divide-and-Conquer sorting algorithms. Both use recursion, but their decomposition methods are different.

Merge Sort divides the array into two approximately equal halves and combines the sorted results using a merge operation.

Quick Sort selects a pivot element and partitions the array into smaller sections based on their relationship with the pivot.

2. Merge Sort

Merge Sort is a recursive sorting algorithm that follows three steps:

1. Divide the array into two halves.
2. Recursively sort both halves.
3. Merge the sorted halves into one sorted array.

For example, consider:

[A=[38,27,43,3,9,82,10,15]]

First, divide the array:

Left half	**Right half**
38, 27, 43, 3	9, 82, 10, 15

Divide again:

Subproblem	**Elements**
A1	38, 27
A2	43, 3
A3	9, 82

A4	10, 15

Continue dividing until each subarray contains one element.

Then merge the smaller sorted arrays.

Merge stage	**Result**
First merge	27, 38
Second merge	3, 43
Third merge	9, 82
Fourth merge	10, 15
Final merge	3, 9, 10, 15, 27, 38, 43, 82

Final sorted array:

[[3,9,10,15,27,38,43,82]]

3. Merge Sort Pseudocode

ALGORITHM MergeSort(A, low, high)

INPUT:

- A – array to be sorted.
- low – starting index.
- high – ending index.

OUTPUT:

- Array sorted in ascending order.

BEGIN

1. IF low ≥ high THEN
    - RETURN.
2. END IF

3. mid ← FLOOR((low + high) / 2)
4. MergeSort(A, low, mid)
5. MergeSort(A, mid + 1, high)
6. Merge(A, low, mid, high)
7. RETURN sorted array.

END

4. Merge Operation

The merge operation combines two already sorted subarrays.

It compares the first available element of each subarray and copies the smaller element into the output section.

The process continues until all elements have been copied.

Merge Sort has a time complexity of:

[boxed{O(nlog n)}]

It also requires additional temporary storage, generally \(O(n)\), for the merging process.

QUICK SORT AND PARTITION QUALITY

1. Introduction to Quick Sort

Quick Sort is a Divide-and-Conquer sorting algorithm that uses a pivot element to partition an array.

The pivot is selected from the array. The partition operation rearranges the elements so that:

- Elements smaller than or equal to the pivot are placed on one side.
- Elements greater than the pivot are placed on the other side.
- The pivot reaches its partition position.

The algorithm then recursively sorts the two partitions.

2. Example of Quick Sort

Consider:

[A=[8,3,1,7,0,10,2]]

Choose the last element, 2, as the pivot.

After partitioning:

[[1,0,2,7,3,10,8]]

The pivot 2 is placed in its correct position. The left section contains smaller elements, and the right section contains larger elements.

The algorithm recursively sorts both sections.

Final sorted array:

[[0,1,2,3,7,8,10]]

3. Quick Sort Pseudocode

ALGORITHM QuickSort(A, low, high)

INPUT:

- A – array to be sorted.
- low – starting index.
- high – ending index.

OUTPUT:

- Array sorted in ascending order.

BEGIN

1. IF low less than high THEN
2. pivotPosition ← Partition(A, low, high)
3. QuickSort(A, low, pivotPosition - 1)
4. QuickSort(A, pivotPosition + 1, high)
5. END IF
6. RETURN sorted array.

END

4. Partition Pseudocode

ALGORITHM Partition(A, low, high)

INPUT:

- A – array.
- low – starting index.
- high – ending index.

OUTPUT:

- Final position of the pivot.

BEGIN

1. pivot ← A[high]
2. i ← low - 1
3. FOR j ← low TO high - 1 DO
4. IF A[j] ≤ pivot THEN
    - i ← i + 1
    - Exchange A[i] and A[j].
5. END IF
6. END FOR
7. Exchange A[i + 1] and A[high].
8. RETURN i + 1.

END

5. Balanced and Unbalanced Partitions

The performance of Quick Sort depends heavily on the quality of the partitions.

A balanced partition divides the array into two sections of approximately equal size.

An unbalanced partition produces one large section and one very small section.

Partition type	Example split for 100 elements	Effect
Balanced	50 and 50	Produces a relatively shallow recursion tree
Moderately unbalanced	70 and 30	Requires more recursion levels
Severely unbalanced	90 and 10	Produces a deeper recursion tree
Extreme	99 and 1	Can lead to worst-case behaviour

6. Quick Sort Complexity

Case	Time complexity	Explanation
Best case	\(O(n\log n)\)	Partitions are balanced.
Average case	\(O(n\log n)\)	Partitions are reasonably balanced on average.
Worst case	\(O(n^2)\)	Partitions repeatedly contain one very small and one very large section.

7. Why Decomposition Quality Matters

Merge Sort always divides the array into approximately equal halves. Therefore, its recursion structure is predictable.

Quick Sort depends on pivot selection. If the pivot repeatedly creates unbalanced partitions, the recursion depth increases and the total work may become quadratic.

This demonstrates that the quality of decomposition has a major effect on algorithm performance.